10

Лекция 13–14 апреля

Универсальная машина

Константин Чухарев

Дискретная математика — Весна 2027

Варианты и устойчивость

> *Вопрос, могут ли машины думать, примерно так же актуален, как вопрос, умеют ли подводные лодки плавать.*
>
> ***— Эдсгер Дейкстра***

Произвол в определении

В семёрке машины легко заподозрить произвол.

Таблица показывает, что каждую альтернативу обсуждали всерьёз.

В определении	**Разумная альтернатива**
лента в обе стороны	односторонняя лента, бесконечная вправо
сдвиг на клетку	сдвиг на месте
одна лента	k лент с отдельными головками
детерминизм	недетерминированный выбор перехода

Если бы класс вычислимого зависел от этих деталей, слово «вычислимо» описывало бы одну конструкцию, а не понятие.

Поэтому эквивалентность вариантов проверяют прежде, чем строить на модели теорию.

Многоленточная машина

Программисту привычны несколько массивов данных, и k лент выглядят как удобство, а не как новая сила.

Определение 1 (Многоленточная МТ)

k лент и k головок: за один шаг каждая головка читает, пишет и сдвигается независимо.

Симуляция выкладывает все ленты на одну колонками: первые ячейки всех лент, за ними вторые, позиции головок помечены маркерами.

Один шаг исходной машины — несколько проходов по занятой части: прочитать символы под маркерами, найти переходы, записать новое и сдвинуть маркеры.

Цена симуляции

Симуляция честная: она повторяет каждый шаг исходной машины, но не бесплатно.

За t шагов каждая головка уходит от старта не дальше чем на t клеток, поэтому занятая часть общей ленты растёт линейно.

После $T(n)$ шагов многоленточной машины занятая часть занимает $O(T(n))$ клеток.

Каждый шаг симуляции обходит её целиком: итог $O(T(n)^2)$ шагов.

Плата за удобство — время, а не сила.

Все оценки в этом модуле — о принципиальной возможности, поэтому квадратичный множитель не меняет выводов.

Недетерминированная машина

Некоторые задачи естественно описываются угадыванием, и квантор существования сидит прямо в определении принятия.

Определение 2 (НМТ)

δ возвращает множество возможных шагов.

Машина принимает вход, если хотя бы одна ветвь вычисления достигает q_{accept}.

Пример (Угадывание делителя)

Машина, распознающая составные числа, получает n и угадывает разложение: на входе 91 ветвь выбирает 7 и 13, перемножает и сравнивает произведение со входом — сравнение сходится, машина принимает.

На простом входе 89 ни одна ветвь не подберёт делителей — принятия не произойдёт ни в одной ветви.

Почему обход в ширину?

Детерминированная симуляция НМТ обходит дерево конфигураций в ширину: все ветви продвигаются по одному шагу.

Глубинный обход рискует навсегда уйти в бесконечную ветвь и пропустить принимающую, которая сидит в соседней.

Ширина дерева ограничена: из вершины выходит не более b переходов, поэтому на глубине t не более b^t вершин.

При $b = 2$ и глубине 10 обход держит в работе $2^{10} = 1024$ ветви прежде, чем пойти на глубину 11.

Плата — экспонента, но для вопроса «принимает ли» важна возможность, а не скорость.

Устойчивость модели

Четыре строки таблицы — четыре кандидата в определение, и каждый симулируется базовой машиной.

Теорема 1 (Эквивалентность вариантов)

Односторонняя лента, сдвиг на месте, k лент и недетерминизм вычисляют тот же класс языков, что и одноленточная детерминированная МТ.

То же верно для двухстековых и счётчиковых машин Минского и регистровых машин.

Симуляция вариантов

Доказательство

Докажем симуляцией: для каждой модификации опишем базовую машину, воспроизводящую её работу шаг за шагом.

Одностороннюю ленту укладываем двумя дорожками: правую половину по верхней, левую — отражённой по нижней, границу метит нулевая клетка.

Сдвиг на месте заменяется парой шагов через свежее состояние: влево и обратно вправо.

Несколько лент выкладываются колонками с маркерами головок.

Двухстековая машина хранит ленту в двух стеках, а машины Минского — в счётчиках и регистрах.

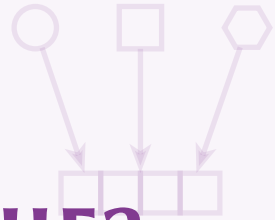
Недетерминизм обходится поиском в ширину по дереву конфигураций.

Что доказывает эквивалентность

> Сила модели определяется задачами, а не деталями конструкции.
>
> О вычислимости можно говорить, не уточняя версию машины.

Дальше слово «машина» свободно означает любую из версий.

Доказательства ведутся на той, где меньше техники, а вывод переносится на все остальные.



Тезис Чёрча–Тьюринга

> *Впервые удалось дать абсолютное определение интересного эпистемологического понятия — понятия, не зависящего от выбранного формализма.*
>
> **— Курт Гёдель**

Вычислимая функция

Ответ машины не ограничен словом «да» или «нет»: на ленте может остаться новое слово.

Определение 3 (Вычисление функции)

МТ **вычисляет** функцию $f : \Sigma^* \mapsto \Sigma^*$, если на входе w она останавливается с $f(w)$ на ленте в точности тогда, когда $f(w)$ определено.

Функция может быть частичной: где не определена — машина не останавливается вовсе.

Функция, определённая на всех словах, называется всюду определённой (*total*).

Частичность на машинах

Различие проще увидеть на двух машинах, чем запомнить по определению.

Пример (Всюду определённая и частичная)

Инверсия битов вычисляет всюду определённую функцию: машина проходит строку до пробела и останавливается, $f(001) = 110$.

Половина унарной длины — частичная функция: на 1^6 машина вычёркивает три пары и останавливается с 1^3, а на 1^5 пара не сходится — машина зацикливается, и $f(1^5)$ не определено.

Частичность — отсутствие остановки, а не особый ответ машины.

На нечётной длине машина не сообщает ошибку — она просто не останавливается.

Тезис

Определение вычислимой функции выбрано, но выбор требует обоснования: почему мера вычислимого — именно машина.

Тезис Чёрча–Тьюринга: класс функций, вычислимых на МТ, совпадает с классом интуитивно вычислимых функций.

Практическое правило: если задача неразрешима на МТ, она неразрешима нигде.

Ни на другом языке, ни на другой архитектуре, ни на суперкомпьютере.

Чем тезис не является?

Тезис — не теорема.

Теорема связывает два формальных объекта, а «интуитивно вычислимая функция» формальна не больше, чем наши представления о любом конечном процессе.

Статус другой: принятое определение понятия «алгоритм», проверяемое опытом.

Проверка идёт с двух сторон.

Все независимые формализмы сходятся к одному классу, а процедура, вычисляющая что-то вне его, за девяносто лет не предъявлена.

Три формализма 1936 года

Одна и та же граница вычислимого найдена трижды и независимо:

Формализм	**Чем торгует**
Машина Тьюринга	лента, головка, состояния
Лямбда-исчисление Чёрча	подстановка вместо ленты
Общерекурсивные функции Гёделя	композиция и минимизация вместо переходов

Гёделя убедила именно механическая наглядность МТ: лента и клетки — буквально человек с бумагой.

Позже тот же класс дали клеточные автоматы и другие модели — счёт формализмов растёт, класс один.

Универсальная машина

> *Лучший способ предсказать будущее — изобрести его.*
>
> ***— Алан Кэй***

Сколько существует машин?

Каждая МТ — конечная строка символов: её таблица переходов записывается текстом.

Описаний длины не более n символов конечное число, а всякое описание имеет какую-то длину, поэтому описаний счётное число — счётное объединение конечных множеств.

Машин $\aleph_0$, а языков $2^{\aleph_0}$ — континуум.

Значит, почти все языки не распознаваемы никакой машиной: невычислимого неизмеримо больше, чем вычислимого.

Для масштаба: при алфавите кодировки из 8 символов описаний длины ровно 60 не больше $8^{60} < 10^{55}$ — конечное число, пусть и астрономическое.

Конечность на каждой длине, счётность в сумме — вот весь аргумент.

Кодирование машины

Таблица переходов — конечный текст: состояния, символы и направления нумеруются, каждый переход становится пятёркой (q, a, q', b, d).

Проведём через код машину инверсии битов: одно рабочее состояние q_0, одно принимающее q_{accept} и три правила.

Кодировка: состояния $q_0 \to 1$ и $q_{\text{accept}} \to 2$, символы $0 \to a$, $1 \to b$, пробел $\to c$, направления $L \to l$ и $R \to r$.

Пример (Таблица переходов в пятёрках)

Правило	Код
$(q_0, 0) \to (q_0, 1, R)$	$(1, a, 1, b, r)$
$(q_0, 1) \to (q_0, 0, R)$	$(1, b, 1, a, r)$
$(q_0, \text{пробел}) \to (q_{\text{accept}}, \text{пробел}, R)$	$(1, c, 2, c, r)$

Код как строка

Каждая строка таблицы уже пятёрка, и вся машина собирается в одну строку без пробелов.

$$\langle M \rangle = (1, a, 1, b, r)(1, b, 1, a, r)(1, c, 2, c, r),$$

Пятёрки идут подряд без разделителей: каждая ровно одиннадцать символов, поэтому чтение кода не оставляет неоднозначности.

Тридцать три символа — и машину инверсии можно положить на ленту как вход.

> Программа — те же символы, которые машина читает как данные.
>
> Различие программы и данных — соглашение о роли строки.

Теорема об универсальной машине

Код машины — строка, а машина умеет читать строки: значит, одна машина может исполнять все.

Теорема 2 (Универсальная МТ)

Существует машина U, которая по кодировке $\langle M \rangle$ и строке w симулирует работу M на w.

U принимает, отвергает и зацикливается ровно тогда, когда это делает M.

Набросок: три дорожки на ленте — программа, текущее содержимое ленты M, текущее состояние с маркером головки.

Каждый шаг U: найти в таблице подходящую пятёрку и обновить дорожки — написание эмулятора на ассемблере.

Прогон на конкретном коде

Пример (Шаги U на паре $\langle M \rangle$ и 001)

На ленте универсальной машины лежат код машины инверсии, разделитель и входное слово 001, на дорожке состояния записана единица, маркер стоит на первом символе входа.

Состояние 1 и символ a под маркером — подходит первая пятёрка $(1, a, 1, b, r)$: записать b, сдвинуться вправо.

Следующие два шага инвертируют оставшиеся символы: a меняется на b, затем b на a, а после трёх инверсий под маркером разделитель c — пятёрка $(1, c, 2, c, r)$ переводит U в принимающее состояние 2.

На дорожке работы остаётся 110 — ровно то, что оставила бы сама машина инверсии.

Различие одно: U не знает таблицу заранее и ищет пятёрку в коде, поэтому один шаг M обходится U в несколько собственных шагов.

Хранимая программа

Универсальная машина Тьюринга — 1936 год.

Архитектура фон Неймана, где программа и данные живут в общей памяти, — 1945 год.

> В телевизоре и в стиральной машине стоит одна и та же электронная схема — различает их только программа.
>
> Одна машина исполняет любую программу — в этом замысел универсальной машины.

Интерпретаторы

Универсальная МТ — прообраз интерпретатора: программы, читающей и исполняющей другие программы.

- виртуальные машины — Java, .NET, WebAssembly
- эмуляторы процессоров
- шеллы и REPL.

Каждый интерпретатор держит чужую программу как данные и крутит цикл «прочитать шаг — исполнить шаг» — ровно конструкция U.

> Компилятор переводит текст заранее, интерпретатор исполняет на лету — оба читают программу как данные.
>
> Разница в моменте исполнения, а не в силе.

Куайн

Раз программы суть строки, найдётся машина, печатающая собственный текст: такая программа называется *квайном* (*quine*).

Пример (Куайн из шаблона и подстановки)

Лобовая попытка — вписать в программу её текст — буксует: копия обёрнута в команду печати и короче целого. Выход — разделить роли: данные хранят шаблон с маркером, код печатает шаблон с вклеенной цитатой.

```
s = "s = <>\nprint(subst(s, quote(s)))"
print(subst(s, quote(s)))
```

Подстановка вклеивает цитату вместо маркера, и на печать выходят в точности две строки программы. Тем же механизмом самоссылки держится диагональное доказательство неразрешимости.

Программы о программах

Программа, анализирующая программу, — обычное дело:

- антивирус решает, опасен ли файл
- статический анализатор ищет ошибки без запуска
- верификатор доказывает свойства кода.

Вопрос «что сделает эта программа с этой?» — центральный вопрос всей теории вычислимости.

Проверка свойств схем сводится к SAT.

На машинах Тьюринга ответ окажется отрицательным: точного анализатора не существует.

Что сделает машина с входом?

Точная постановка: дана пара $(\langle M \rangle, w)$ — код машины и вход.

Исходов три: M принимает w, отвергает или зациклится — и задача в том, чтобы предсказать исход по паре.

U умеет исполнять: запустила симуляцию — и повторяет шаги M.

Уметь исполнять — не уметь предсказывать: пока M не остановилась, симуляция не отличает «ещё работает» от «не остановится никогда».

Для остановившихся машин ответ приходит сам, вся трудность сидит в зацикливании: ответ «нет» симуляция не сообщает.

Предсказателя, который отвечает на этот вопрос всегда и верно, не существует — диагональный аргумент Кантора вернётся против самих машин.

Что мы поняли?

Варианты машины эквивалентны, и тезис закрепил роль меры вычислимого.

Универсальная машина превратила программы в данные и подвела к теории неразрешимости.

- Варианты машины симулируются базовой: сила одна, разное только время.
- Тезис Чёрча–Тьюринга — принятое определение вычислимости, а не теорема: одна из сторон неформальна.
- Машин счётное число, языков — континуум: невычислимого больше, чем вычислимого.
- Универсальная машина исполняет любую программу: кодирование — строка пятёрок, программа — данные.
- Исполнение — ещё не предсказание.

Вопросы для самопроверки

Вопросы для самопроверки — об универсальной машине

1. Почему многоленточная МТ не сильнее одноленточной и откуда берётся квадратичный множитель симуляции?
2. Почему симуляция недетерминированной МТ идёт в ширину и сколько ветвей ждёт обход на глубине 10 при ветвлении 2?
3. Сформулируйте тезис Чёрча–Тьюринга и объясните, почему он не теорема.
4. У машины пустая таблица переходов и $q_0 = q_{\text{accept}}$. Сколько пятёрок в её коде и что она делает на любом входе?
5. Почему машин счётное число и что это говорит о языках?
6. Как устроена универсальная машина и что лежит у неё на ленте? Проследите её шаги на коде машины инверсии и входе 01.

Что почитать

Кодирование и универсальная машина — последний материал главы m25.

Чтение закрывает варианты машин, тезис и устройство U.

- m25 — «Устойчивость и варианты», «Тезис Чёрча–Тьюринга», «Универсальная машина Тьюринга».